

OBJECT ORIENTED PROGRAMMING

S.Gayathri

Assistant Professor

Department of Computer Science and Engineering

Jerusalem College of Engineering

UNIT-II: INHERITANCE AND POLYMORPHISM

Inheritance: Single, Multi-Level, Multiple, Hierarchical and Hybrid – Abstract Classes - Polymorphism- Function Overloading: Overloading Constructor Functions - Copy Constructors-Operator Overloading: Unary Operators – Binary operators

INHERITANCE

- The mechanism of deriving a new class from an old class is called inheritance
- The old class → base class
- The new class → derived class
- A class can inherit from one or more classes. Reusability is an important feature of OOP
- A derived class can be defined by specifying its relationship with the base class in addition to its own details

Defining Derived Class

```
class derived-class-name : visibility-mode base-class-name
{
.....
.....
}
```

The colon indicates that the derived class name is derived from base-class-name. The visibility mode is optional and if present, may be either **private or protected or public**. The default is private.

Example

```
class ABC : private XYZ // private derivation
{
```

```
    members of ABC;
```

```
};
```

```
class ABC : public XYZ // public derivation
```

```
{
```

```
    members of ABC;
```

```
};
```

```
class ABC : protected XYZ // protected derivation
```

```
{
```

```
    members of ABC;
```

```
};
```

```
class ABC : XYZ // private by default
```

```
{
```

```
    members of ABC
```

```
};
```

Note:

- Base class is privately inherited by the derived class—> public members of base class become private members of derived class.
- Base class is publicly inherited by the derived class—> public members of base class become public members of derived class.

- When a base class is privately inherited by a derived class, public members of the base class can only be accessed by the member functions of the derived class. Private members of base class are inaccessible to the objects of the derived class
- When a base class is protected inherited by a derived class, public members of the base class can only be accessed by the member functions of the derived class. Private members of base class are inaccessible to the objects of the derived class. If private members of base class are to be inherited to derived class then declare them as protected
- When the base class is publicly inherited, public members of the base class become public members of the derived class and therefore they are accessible to the objects of the derived class. In both the cases, the private members are not inherited and therefore, the private members of a base class will never become the members of its derived class

- In inheritance, some of the base class data elements and member functions are “inherited” into the derived class. We can add our own data and member functions and thus extend the functionality of the base class. Inheritance, when used to modify and extend the capability of the existing classes, becomes a very powerful tool for incremental program development

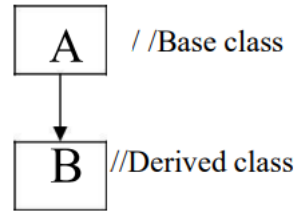
Base Class Visibility	Derived class visibility			
	Public Derivation	Private Derivation	Protected Derivation	
Private	Not inherited	Not inherited	Not inherited	
Protected	Protected	Private	Protected	
Public	Public	Private	Protected	

Types of Inheritance:

1. Single Inheritance
2. Multi level Inheritance
3. Multiple Inheritance
4. Hybrid inheritance
5. Hierarchical Inheritance.

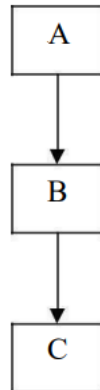
1.Single Inheritance:

one derived class inherits from only one base class. It is the most simplest form of Inheritance.

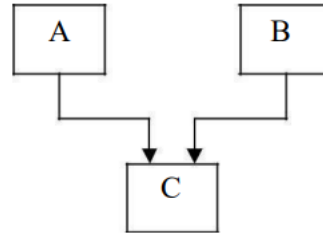


2.Multilevel Inheritance:

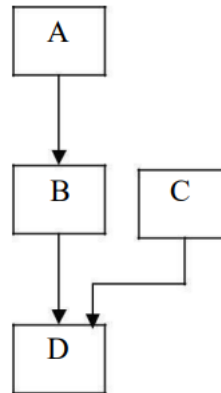
In this type of inheritance the derived class inherits from a class, which in turn inherits from some other class. The Super class for one, is sub class for the other.



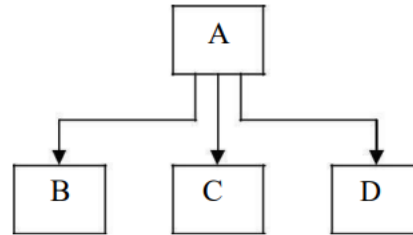
3. Multiple Inheritance: In this type of inheritance a single derived class may inherit from two or more than two base classes.



4. Hybrid Inheritance: Hybrid inheritance is combination of two or more inheritances such as single, multiple, multilevel or Hierarchical inheritances.



5.Hierarchical Inheritance: Inheriting is a method of inheritance where one or more derived classes is derived from common base class.



Single Inheritance (Public)

```
class B
{
int a; // Private: Not inheritable
public:
int b;
void get_ab( );
int get_a(void);
void show_a(void);
};
```

```
class D :public B
{
int c;
Public:
        void mul(void);
        void display(void);
};
Void B:: get_ab(void)
{ a = 5; b = 10; }
Int B::get_a()
{ return a; }
Void B::show_a()
{ cout << "a=" << a << "\n"; }
```

```
Void D::mul()
{ c = b * get_a(); }
Void D::display()
{ cout << "a=" << get_a() << "\n";
  Cout << "b=" << b << "\n";
  Cout << "c=" << c << "\n"; }
void main()
{
    D d;
    d.get_ab();
    d.mul();
    d.show_a();
    d.display();
    d.b = 20;
    d.mul();
    d.display();
}
```

output:

```
a= 5
a = 5
b = 10
c = 50
a = 5
b = 20
c = 100
```

Single Inheritance (Private)

Class B

```
{ int a;  
  Public:  
  int b;  
  void get_ab();  
  int get_a(void);  
  void show_a(void);  
};
```

Class D: private B

```
{ int c;  
  Public:  
  void mul(void);  
  void display(void);  
};
```

```
Void B::get_ab(void)
{ cout << "Enter values for a and b:";
  cin >> a >> b;
}
Void B::get_a()
{ return a;}
```

```
Void B::show_a()
{ cout << "a=" << a << "\n";
}
```

```
Void D::mul()
{ get_ab();
  c = b * get_a();
}
```

```
Void D::display()
{ show_a();
  Cout << "b=" << b << "\n" << c << "\n";
}
```

```
void main()
{ D d;
  // d.get_ab(); // won't work
  d.mul();
  // d.show_a(); // won't work
  d.display();
  // d.b=20; // won't work
  d.mul();
  d.display();
}
```

Output:

Enter values for a and b : 5 10

a = 5

b = 10

c = 50

Enter values for a and b: 12 20

a = 12

b = 20

c = 240

Making a private member inheritable

- By modifying the visibility limit of the private by public. This will be taking away the advantage of data hiding.
- Third visibility modifier, protected which serve a limited purpose in inheritance.
- A member declared as protected is accessible by the member functions within its class and any class immediately derived from it.

- Class x

```
{  
    Private: //optional  
    .... // visible to member functions  
    Protected: // visible to member functions of its own class and derived classes  
    ...  
    Public: //visible to all functions in the program  
};
```

Multilevel Inheritance

```
class Student
{
    protected:
        int roll_no;
    public:
        void get_number(int);
        void put_number(void);
};

void Student::get_number(int a)
{ roll_no = a; }

void Student::put_number()
{ cout << "Roll number:" << roll_no << "\n":
}
```

```
class test: public Student
```

```
{
```

```
    protected:
```

```
        float sub1;
```

```
        float sub2;
```

```
    public:
```

```
        void get_marks(float, float);
```

```
        void put_marks(void);
```

```
};
```

```
void test::get_marks(float x, float y)
```

```
{ sub1 = x; sub2 = y; }
```

```
void test::put_marks()
```

```
{ cout << "marks in sub1=" << sub1 << "\n" << "marks in sub2=" << sub2 << "\n";
```

```
}
```

```
class result: public test
{
    float total;
    Public:
        Void display(void);
};
void result::display(void)
{
    total = sub1 + sub2;
    put_number();
    put_marks();
    cout << "total=" << total << "\n";
}
```

```
void main()
{
    result student1;
    student1.get_number(111);
    student1.get_marks(75.0,59.5);
    student1.display();
}
```

Output:

Roll number:111

marks in sub1=75

marks in sub2 = 59.5

total = 134.5

MULTIPLE INHERITANCE

```
class M
```

```
{
```

```
protected:
```

```
    int m;
```

```
public:
```

```
    void get_m(int);
```

```
};
```

```
class N
```

```
{
```

```
protected:
```

```
    Int n;
```

```
public:
```

```
    void get_n(int);
```

```
};
```

```
class P : public M, public N
```

```
{
```

```
    public:
```

```
        Void display(void);
```

```
};
```

```
void M::get_m(int x)
{
    m = x; }
void N::get_n(int y)
{
    n = y; }
void P::display(void)
{ cout << "m=" << m;
  cout << "n=" << n;
  cout << "m*n=" << m*n;
}
void main()
{
    P p1;
    P1.get_m(10);
    P1.get_n(20);
    P1.display();
}
```

Output:

```
m=10
n = 20
m*n = 200
```

HYBRID INHERITANCE

Example:

class student

```
{
    protected:
        int roll_no;
    public:
        void get_number(int a)
        {
            roll_no = a;
        }
        void put_number(int a)
        {
            cout << "RollNo:" << roll_no << "\n";
        }
};
```

```
class test: public student
```

```
{
```

```
    protected:
```

```
        float part1, part2;
```

```
    public:
```

```
        void get_marks(float x, float y)
```

```
        {
```

```
            part1 = x; part 2 = y;
```

```
        }
```

```
        void put_marks(void)
```

```
        {
```

```
            cout << "marks obtained:" << "\n";
```

```
            cout << "part1=" << part1 << "\n";
```

```
            cout << "part2=" << part2 << "\n";
```

```
        }
```

```
};
```

```
class Sports
```

```
{
```

```
    protected:
```

```
        float score;
```

```
    public:
```

```
        void get_score(float s)
```

```
        {
```

```
            score = 2;
```

```
        }
```

```
        void put_score(void)
```

```
        {
```

```
            cout << "Spare wt:" << score << "\n";
```

```
        }
```

```
};
```

```
class result : public test, public sports
{
    float total;
    public:
        void display(void);
};

void result::display(void)
{
    total = part1+part2+score;
    put_number();
    put_marks();
    put_score();
    cout << "Total score:" << total << "\n";
}
```

```
int main()
{
    result student_1;
    student_1.get_number(1234);
    student_1.get_marks(27.5,33.0);
    student_1.get_score(6.0);
    student_1.display();
    return 0;
}
```

Output:

Roll No:1234

Marks obtained:

Part1 = 27.5

Part2 = 33

Sparts wt: 6

Total Score : 66.5

HIERARCHICAL INHERITANCE

```
#include <iostream.h>
class Shape
{
    protected:
        float area;
        float perimeter;
    public:
        void display()
        {
            cout << "Area is " << area << endl;
            cout << "Perimeter is " << perimeter << endl;
        }
};
```

```
class Circle : public Shape
{
    private:
        float radius;
    public:
        void get_data()
        {
            cout << "Enter radius" << endl;
            cin >> radius;
        }
        void calc_area()
        {
            area = 3.14 * radius * radius;
        }
        void calc_perimeter()
        {
            perimeter = 2 * 3.14* radius;
        }
};
```

```
class Rectangle : public Shape
{
    private:
        float width, height;
    public:
        void get_data()
        {
            cout << "Enter width" << endl;
            cin >> width;
            cout << "Enter height" << endl;
            cin >> height;
        }
        void calc_area()
        {
            area = width * height;
        }
        void calc_perimeter()
        {
            perimeter = 2 * ( width + height);
        }
}
```

```
int main()
{
    Circle c;
    Rectangle r;
    c.get_data();
    r.get_data();
    c.calc_area();
    c.calc_perimeter();
    r.calc_area();
    r.calc_perimeter();
    c.display();
    r.display();
}
```

Output:

Enter radius

5

Enter width

12

Enter height

17

Area is 78.539818

Perimeter is 31.415926

Area is 204

Perimeter is 58

ABSTRACT CLASS

- An abstract class is a class which is mostly used as a base class.
- It contains at least one pure virtual function.
 - Abstract classes can be used to specify an interface that must be implemented by all subclasses.
- The virtual function in the base class is made pure, when there is no function definition relative to its class and which is to be redefined in the derived classes.
- We cannot create an instance or object for an abstract class.

Syntax:

```
virtual void display()=0;
```

Program:

```
class shape
{ public:
virtual void area()=0;
};
class circle:public shape
{ public:
void area()
{ cout<<"Circle area\n"; }
};
class rectangle:public shape
{ public:
void area()
{ cout<<"Rectangle area\n"; }
};
```

```
void main()
{
    clrscr();
    shape *sptr;
    circle c1;
    sptr=&c1;
    sptr->area();
    rectangle r1;
    sptr=&r1;
    sptr->area();
    getch();
}
```

Output:

Circle area

Rectangle area

Explanation:

- The class shape act as an abstract class.
- It specifies an interface which is used by all the derived classes.
- Thus it is never used directly it simply gives skeleton to other derived classes. It contains one pure virtual function area().

Polymorphism

Polymorphism: Taking more than one form

Types:

(i) Runtime polymorphism

Example:

Virtual Function

(ii) Compile time polymorphism

Example:

- Function overloading
- Operator overloading

Function Overloading

More than one function with same name but with different arguments

Example:

```
int volume(int);  
double volume(double, int);  
long volume(long, int, int);  
int main()  
{  
    cout << volume(10) << endl;  
    cout << volume(2.5, 8) << endl;  
    cout << volume(100L,-15,15) << endl;  
    return 0;  
}
```

```
int volume(int s)
{
    return (s * s * s);
}
double volume(double r, int h)
{
    return (3.14 * r * r * h);
}
long volume(long l, int b, int h)
{
    return (l * b * h);
}
```

Output:

100

157

112500

Function Overloading: Overloading Constructor Functions

```
class complex
{
float x,y;
Public:
complex()
{ }
complex(float a)
{
x=y=a;
}
```

```
complex(float real, float imag)
{
X=real;
Y=imag;
}
friend complex sum(complex,complex);
friend void show(complex);
};
complex sum(complex c1,complex c2)
{
complex c3;
c3.x=c1.x+c2.x;
c3.y=c1.y+c2.y;
Return(c3);
}
```

```

Void show()
{
    Cout<<c.x<<" +j"<<c.y<<endl;
}
Int main()
{
    complex A(2.7,3.5)
    complex B(1.6)
    complex C;
    C=sum(A,B);
    cout<<"A="<<show(A);
    cout<<"B="<<show(B);
    cout<<"C="<<show(C);
    return 0;
}

```

Ouput:

A=2.7 +j 3.5

B=1.6 +j 1.6

C=4.3 +j 5.1

Operator overloading:

- Ability to provide operators with a special meaning for a data type. The mechanism of giving such a special meaning to an operator is known as operator overloading.
- **We cannot overload the following operators.**
 - Class member access operators (., *)
 - Scope resolution operator (::)
 - Sizeof operator(sizeof)
 - Conditional operator (?:)

Defining operator overloading:

This is done with a help of a special function called operator function.

Syntax:

```
return type classname:: operator op (arg)
{
    Body of the function;
}
```

Operator op → function

Op → operator to be overloaded

The process of overloading involves the following steps:

1. Create a class that defines the datatype that is to be used in the overloading operation.
2. Declare the operator function **operator op()** in the public part of the class. It may be either member function or friend function.
3. Define the operator function to implement the required operation.

Overloading in unary operator

class space

```
{  
    int x, y, z;  
    public:  
        void get_data(int a, int b, int c);  
        void operator –();  
        void display(void);  
};
```

void space::get_data(int a, int b, int c)

```
{  
    x = a;  
    y = b;  
    z = c;  
}
```

```
void space::operator – ()
```

```
{  X = -x;
```

```
    Y = -y;
```

```
    Z = -z;  }
```

```
void space::display()
```

```
{  cout << x << endl;
```

```
    cout << y << endl;
```

```
    cout << z << endl; }
```

```
int main()
```

```
{  space s1;
```

```
    s1.get_data(10,-20,30);
```

```
    s1.display();
```

```
    -s1;
```

```
    s1.display();
```

```
    return 0;
```

```
}
```

Output:

10 -20 30

-10 20 -30

Overloading Binary operator:

```
class complex
```

```
{
```

```
    float x;
```

```
    float y;
```

```
    public:
```

```
        complex() { }
```

```
        complex(float real, float imag)
```

```
        {
```

```
            X = real;
```

```
            Y = imag;
```

```
        }
```

```
        complex operator + (complex);
```

```
        void display();
```

```
};
```

```
complex complex::operator + (complex c)
```

```
{
```

```
    complex temp;
```

```
    temp.x = x + c.x;
```

```
    temp.y = y + c.y;
```

```
    return temp;
```

```
}
```

```
void complex::display()
```

```
{
```

```
    cout << x << "+i" << y;
```

```
}
```

```
int main()
{
    complex c1, c2, c3;
    c1 = complex(2.5,3.5);
    c2 = complex(1.6,2.7);
    c3 = c1+ c2;
    cout << "c1=" << c1.display();
    cout << "c2=" << c2.display();
    cout << "c3=" << c3.display();
    return 0;
}
```

Output:

c1=2.5+i3.5

c2=1.6+i2.7

c3=4.1+i6.2